TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PURWANCHAL CAMPUS

A MAJOR PROJECT REPORT ON

A VISION-BASED BALL BALANCING PLATFORM USING OPENCV AND PID CONTROL

SUBMITTED BY:

BISHAKHA POKHREL(49210)

RUDRA KHATRI(49230)

SNEHA YADAV(49238)

SUSANT DAHAL (49243)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PURWANCHAL CAMPUS

DHARAN, NEPAL

30 MARCH, 2026

A MAJOR PROJECT REPORT ON A VISION-BASED BALL BALANCING PLATFORM USING OPENCV AND PID CONTROL

Submitted By:

BISHAKHA POKHAREL(49210)

RUDRA KHATRI(49230)

SNEHA YADAV(49238)

SUSANT DAHAL (49243)

Project Supervised by:

Bishnu Choudhary

Submitted To:

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering

Tribhuvan University

Dharan, Nepal

30 March, 2026

COPYRIGHT©

DECLARATION

We declare that the work hereby submitted for the Bachelor of Engineering in Computer Engineering at the Institute of Engineering, Purwanchal Campus entitled **"A VISION-BASED BALL BALANCING PLATFORM USING OPENCV AND PID CONTROL"** is our own work and has not been previously submitted at any university for any academic award.

We authorize the Institute of Engineering, Purwanchal Campus, to lend this report to other institutions or individuals for the purpose of scholarly research.

Group Members:

Bishakha Pokharel[49210]

Rudra Khatri [49230]

Sneha Yadav [49238]

Susant Dahal [49243]

Date: March, 2026

ACKNOWLEDGEMENT

We would like to express our sincere appreciation to the academic community for their valuable support and guidance during the making of this project. The constructive feedback, insightful suggestions, and encouragement received throughout this process have played a vital role in refining our ideas and enhancing the overall quality of the work.

We are especially grateful to our supervisor, **Mr. Bishnu Choudhary**, for his constant guidance, expertise, and encouragement, which have been instrumental in shaping the direction and execution of this project.Similarly,we would like to extend our regards to Er.Manoj Kumar Guragain, H.O.D. Electronics and Computer department.

We also acknowledge the collaborative and intellectually stimulating environment provided by the department, which has significantly contributed to the development of this project. The collective commitment to academic excellence and innovation has been a constant source of inspiration.

This work is the result of shared knowledge, thoughtful discussions, and continuous support, for which we are truly thankful.

BISHAKHA POKHREL (PUR078BEI010)

RUDRA KHATRI (PUR078BEI031)

SNEHA YADAV (PUR078BEI040)

SUSANT DAHAL (PUR078BEI046)

ABSTRACT

This project presents the design and implementation of a vision-based ball balancing platform using OpenCV and PID control. The system utilizes a mobile phone camera to capture real-time video, which is processed to detect the ball position. The positional error is transmitted to an ESP32 microcontroller, where a PID controller generates corrective signals to actuate stepper motors.The system operates as a closed-loop control system and achieves a settling time of approximately 2.8 seconds, with an overshoot of 8.5% .The system achieves stable performance with bounded steady-state error. A small offset is observed due to practical limitations such as camera alignment, mechanical constraints, and real-time processing delay. The system demonstrates stable and responsive behavior under varying conditions.The project highlights a low-cost and effective integration of computer vision and embedded control systems for real-time stabilization applications.

Keywords: *Computer Vision, PID Control, ESP32, OpenCV, Ball Balancing, Real-Time Control*

TABLE OF CONTENTS

COPYRIGHT i

DECLARATION ii

ACKNOWLEDGEMENT iii

ABSTRACT iv

LIST OF FIGURES ix

LIST OF TABLES x

LIST OF ABBREVIATIONS xi

1 INTRODUCTION 1

1.1 Background 1

1.2 Gap Identification 1

1.3 Motivation 1

1.4 Objectives 1

1.5 Scopes 2

2 RELATED THEORY 3

2.1 Hardware Used 3

2.2 Software Tools and Libraries Used 5

2.2.1 Embedded Development 5

2.2.2 Vision Processing and Monitoring 5

2.3 Core Functionality 5

2.4 Computer Vision 6

2.5 PID Control 6

2.6 PID Parameter Tuning 6

2.7 Stepper Motor Control 7

2.8 Real-Time Control Loop 7

3 LITERATURE REVIEW **8**

4 METHODOLOGY **10**

4.1 Block Diagram of Model 10

4.2 System Overview 11

4.3 System Architecture 11

4.4 Vision-Based Object Detection 12

4.5 PID Control Implementation 13

4.6 Stepper Motor Control 14

4.7 Implemented Real-Time Control Loop 14

4.8 Experimental Validation 15

4.9 Hardware Wiring and Integration 15

4.10 System Implementation 16

5 RESULTS AND DISCUSSION **17**

5.1 System Implementation Results 17

5.2 Performance Metrics 17

5.2.1 Settling Time 17

5.2.2 Overshoot 17

5.2.3 Steady-State Error 18

5.3 Performance Evaluation 18

5.3.1 Settling Time 18

5.3.2 Steady-State Error 18
5.3.3 Overshoot 18
5.3.4 Stability 18
5.3.5 Experimental Results 18
5.3.6 Graphical Analysis 19
5.3.7 Vision System Performance 21
5.4 Control System Performance 21
5.4.1 Hardware Performance 22
5.5 System Limitations Observed 22
5.6 Challenges Faced 22
6 CONCLUSION **23**
7 FUTURE ENHANCEMENT **24**
7.1 Advanced Control Techniques 24
7.2 AI-Based Vision System 24
7.3 Hardware and System Optimization 24
REFERENCES
APPENDIX

LIST OF FIGURES

Figure 2.1: ESP32 Development Board (Source: Espressif Systems) 3

Figure 2.2: Stepper (Source: STEPPERONLINE) 3

Figure 2.3: Stepper Motor Driver(Source: EASON) 4

Figure 2.4: TFT Display with Rotary Encoder(Source: iFuture Technology) . 4

Figure 2.5: 12V SMPS and Buck Converters(Source: circuitrocks) 4

Figure 4.1: System Block Diagram 10

Figure 4.2: Overall System Architecture 12

Figure 4.3: Flow diagram of the computer vision pipeline 13

Figure 4.4: Detailed hardware wiring diagram 15

Figure 4.5: Real-time system interface 16

Figure 5.1: Large Error Correction 19

Figure 5.2: Fine Tuning 20

Figure 5.3: Normal Stability 20

Figure B.1: System Configuration . . . . . .

Figure B.2: HSV Ball Detection . . . . . .

Figure B.3: Noise Removal . . . . . .

Figure B.4: ROI-Based Detection . . . . . .

Figure B.5: Ball Position Using Moments . . . . . .

Figure B.6: EMA Smoothing . . . . . .

Figure B.7: Error Calculation . . . . . .

Figure B.8: PID Controller . . . . . .
Figure B.9: Serial Communication . . . . . .
Figure B.10Ball Lost Safety . . . . . .
Figure C.1: Serial Data Receiving . . . . . .
Figure C.2: Command Parsing . . . . . .
Figure C.3: Input Limiting . . . . . .
Figure C.4: Inverse Kinematics . . . . . .
Figure C.5: Motor Commanding . . . . . .
Figure C.6: Watchdog Safety . . . . . .
Figure C.7: Main Loop . . . . . .

LIST OF TABLES

Table 5.1: System Performance Results 19

Table A.1: Revised Project Budget . . . . . .

LIST OF ABBREVIATIONS

CV	Computer Vision
Dev Board	Development Board
ESP32	Microcontroller
I^2C	Inter-Integrated Circuit
LCD	Liquid Crystal Display
PID	Proportional–Integral–Derivative
PWM	Pulse Width Modulation
UART	Universal Asynchronous Receiver–Transmitter
Wi-Fi	Wireless Fidelity

CHAPTER 1

INTRODUCTION

1.1 Background

Traditional stabilization systems rely on sensors such as gyroscopes and accelerometers, which provide limited information about object position. Vision-based systems offer richer environmental feedback and enable more accurate tracking. This project presents a system that integrates computer vision with embedded control to achieve real-time stabilization of a ball on a tilting platform.

1.2 Gap Identification

Most existing stabilization systems rely on sensor-based feedback without visual awareness, limiting accuracy under disturbances. Although vision-based systems exist, they are often complex and expensive. This project addresses the need for a low-cost, practical implementation that combines real-time computer vision with embedded PID control.

1.3 Motivation

Vision-based stabilization is widely used in robotics and automation. The motivation of this project is to develop an affordable and efficient system that demonstrates real-time object stabilization using OpenCV and embedded control techniques.

1.4 Objectives

- To develop a real-time vision-based object stabilization system by integrating OpenCV-based object detection with embedded PID control.
- To evaluate the system's stability and performance under disturbances and assess its effectiveness for practical real-world applications.

1.5 Scopes

The project will include:

- Real-time ball detection and tracking using a camera and OpenCV.
- Implementation of embedded PID control for platform stabilization.
- Integration of motors, drivers, and microcontrollers for precise actuation.
- User interface support for system monitoring and parameter tuning.

CHAPTER 2

RELATED THEORY

2.1 Hardware Used

- ESP32 Development Board (2 units)-One controller handles motor control and PID computation, while the other manages the user interface and communication.

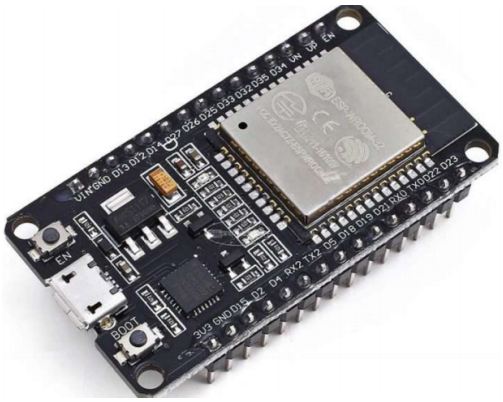

Figure 2.1: ESP32 Development Board (Source: Espressif Systems)

- NEMA 17 Stepper Motors – Provide precise and repeatable platform movement required for accurate stabilization.

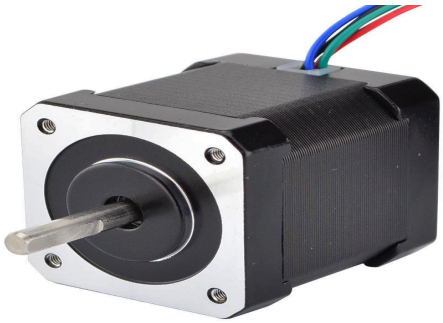

Figure 2.2: Stepper (Source: STEPPERONLINE)

- TB6600 Stepper Motor Drivers – Enable microstepping and high-current control of the stepper motors.

Figure 2.3: Stepper Motor Driver(Source: EASON)

- Mobile Phone Camera (via DroidCam) – Provides real-time video input for object detection and tracking on the PC.
- TFT Display with Rotary Encoder – Allows user interaction, parameter tuning, and real-time system monitoring.

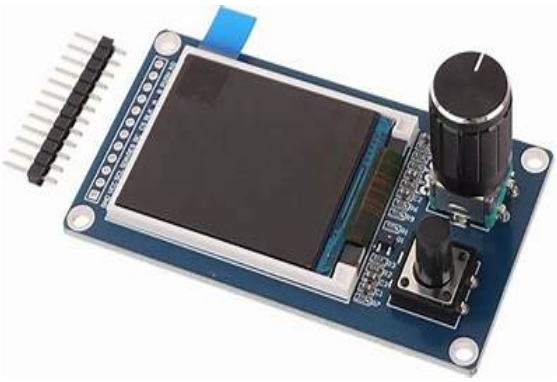

Figure 2.4: TFT Display with Rotary Encoder(Source: iFuture Technology)

- 12V SMPS and Buck Converters – Provide stable and regulated power to motors and logic circuits.

Figure 2.5: 12V SMPS and Buck Converters(Source: circuitrocks)

- Serial Communication (UART) – Used for data exchange between the vision-processing unit and the embedded controller.

2.2 Software Tools and Libraries Used

2.2.1 Embedded Development

- Arduino IDE (ESP32 firmware in C++)
  - Arduino core libraries
  - Stepper motor control libraries
  - HardwareSerial.h
  - math.h

2.2.2 Vision Processing and Monitoring

- Python 3.x (Dashboard)
  - pySerial
  - Matplotlib
  - NumPy
  - OpenCV
  - Time

2.3 Core Functionality

- Vision-based object detection and tracking.
- Embedded PID control for real-time stabilization.
- Precise actuation using stepper motors.
- Real-time monitoring and parameter adjustment through a graphical interface.

2.4 Computer Vision

Computer vision is used to detect and track the ball using a live video feed. The system processes each frame using image processing techniques such as color thresholding and contour detection. The center position of the detected object is calculated and used as feedback for the control system.

2.5 PID Control

A PID controller is implemented to minimize the error between the desired and actual position of the ball. The proportional term reacts to the current error, the integral term removes accumulated error, and the derivative term predicts future trends. This ensures stable and smooth control of the platform. The control law is given by:

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt} \tag{2.1}$$

where:

- $e(t)$ is the error between desired and actual position
- K_p is proportional gain
- K_i is integral gain
- K_d is derivative gain

The error is defined as:

$$e_x = x_{ball} - x_{center} \tag{2.2}$$

$$e_y = y_{ball} - y_{center} \tag{2.3}$$

2.6 PID Parameter Tuning

PID parameters are tuned experimentally based on system response. Incremental adjustments are made to achieve minimal oscillation, reduced steady-state error, and faster

response time.

2.7 Stepper Motor Control

Stepper motors are controlled using discrete pulses generated by the ESP32. The required platform tilt is converted into motor steps. Microstepping is used to achieve smooth motion and higher positional accuracy.

2.8 Real-Time Control Loop

The system operates as a continuous closed-loop system. The camera captures the ball position, which is processed to compute error. The PID controller generates corrective action, which is applied through stepper motors to adjust the platform.

CHAPTER 3

LITERATURE REVIEW

Vision-based ball balancing systems have gained significant attention in the fields of robotics, control engineering, and computer vision due to their applications in automation, real-time control, and intelligent robotic systems. These systems combine image processing techniques with closed-loop control algorithms to achieve stable balancing of a ball on a movable platform.

Bradski and Kaehler [1] introduced the fundamentals of OpenCV and demonstrated various computer vision techniques useful for object detection, tracking, and image processing. Their work provides the foundation for implementing real-time ball tracking in vision-based balancing systems.

Ogata [2] explained the principles of modern control engineering and PID controllers, which are widely used in balancing applications due to their simplicity and stability. PID control is commonly adopted in ball balancing platforms for maintaining accurate ball positioning.

Zhao et al. [3] reviewed different vision-based control techniques used in ball-balancing robots. The study discussed the integration of cameras, embedded systems, and control algorithms for achieving stable and responsive balancing performance.

Zhang [4] proposed a flexible camera calibration method that improves the accuracy of image-based measurements. Camera calibration plays an important role in determining the exact position of the ball in a vision-controlled balancing platform.

Gonzalez and Woods [5] discussed various digital image processing techniques such as filtering, thresholding, segmentation, and feature extraction, which are essential for accurate object detection and tracking in computer vision systems.

Shi and Tomasi [6] introduced feature tracking methods that improve the robustness of real-time object tracking. Their work is widely used in computer vision applications involving motion analysis and object localization.

Bouguet [7] presented the pyramidal Lucas-Kanade tracking method for efficient feature tracking in video streams. This technique supports real-time object tracking under

varying motion conditions and lighting environments.

Corke [8] explained the integration of robotics, vision systems, and control algorithms for intelligent robotic applications. The study highlighted practical approaches for combining image processing with feedback control systems.

Siciliano and Khatib [9] provided comprehensive knowledge on robotics, including robotic control systems, motion planning, and sensor integration, which are important concepts in designing automated balancing platforms.

Craig [10] discussed robotic mechanics and control techniques relevant to motion control and actuator-based balancing systems. The book also explains kinematics and dynamic control methods used in robotic platforms.

Wang and Chen [11] developed a real-time object tracking and PID-based balancing system using computer vision techniques. Their work demonstrated stable ball positioning and effective real-time response using image-based feedback.

Kirk [12] presented optimal control theory concepts for designing stable dynamic systems. These principles are useful for improving the performance and stability of balancing systems.

Haykin [13] discussed neural networks and learning algorithms that can enhance intelligent control and adaptive balancing systems in future implementations.

Thrun, Burgard, and Fox [14] introduced probabilistic robotics concepts including sensor fusion and localization, which can improve robustness in autonomous balancing systems.

Sutton and Barto [15] explained reinforcement learning approaches that can be applied to adaptive and self-learning balancing systems for improved control performance.

Based on the reviewed literature, it can be observed that combining computer vision with embedded feedback control provides an effective solution for real-time balancing applications. Most existing systems focus either on high-cost industrial solutions or complex implementations. Therefore, this project aims to develop a low-cost and practical vision-based ball balancing platform using OpenCV, ESP32, and PID control while maintaining stable and responsive real-time performance.

CHAPTER 4

METHODOLOGY

4.1 Block Diagram of Model

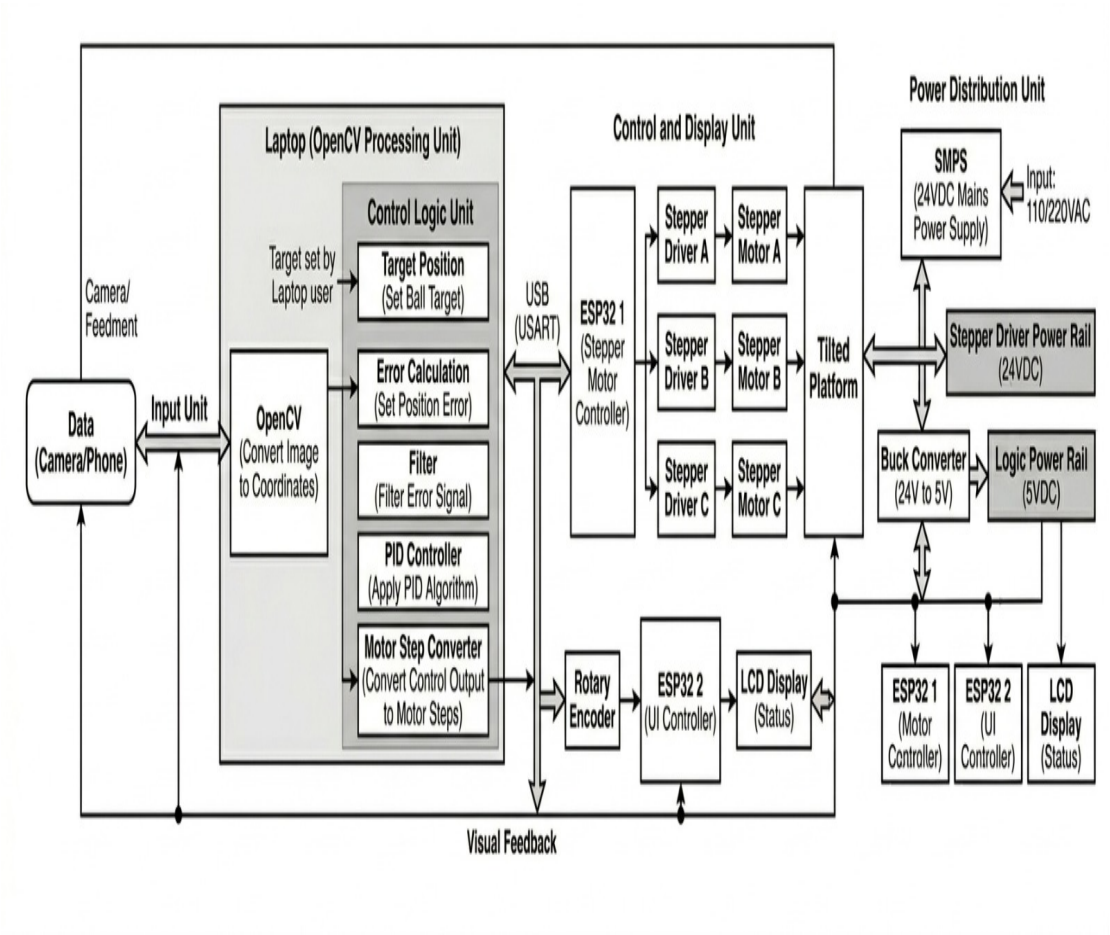

Figure 4.1: System Block Diagram

The system block diagram represents the overall architecture of the vision-based ball balancing platform. A camera captures real-time video of the ball and sends it to the laptop, where OpenCV processes the image and detects the ball position. The detected position is compared with the target position to calculate error, and filtering is applied to reduce noise. A PID controller processes the error and generates corrective motor commands.

These commands are transmitted to the ESP32 motor controller through serial communication. The ESP32 controls the stepper motor drivers and stepper motors, which tilt the platform to balance the ball. A separate ESP32 controller manages the LCD display

and rotary encoder for real-time monitoring and PID parameter adjustment.

The system is powered using a 24V SMPS and a buck converter for regulated power distribution, forming a closed-loop real-time stabilization system.

4.2 System Overview

The proposed system is a vision-based closed-loop control system designed to stabilize a ball on a tilting platform. The system integrates computer vision, embedded control, and electromechanical actuation. The system continuously detects the position of the ball, calculates the deviation from the desired position, and adjusts the platform orientation to minimize the error.

4.3 System Architecture

The system architecture is divided into three main modules:

- Vision Processing Module
- Control and Actuation Module
- User Interaction and Monitoring Module

The vision processing is performed on a PC using OpenCV, where video input from a mobile phone camera (via DroidCam) is used to detect the ball and extract its position coordinates.

The positional data is transmitted to the ESP32 using UART communication. The controller computes the error between the desired and actual ball position and applies a PID controller to generate corrective signals.

These signals are converted into stepper motor commands, which adjust the platform orientation through motor drivers.

The system operates as a closed-loop control system, where continuous visual feedback is used to update control actions in real time and maintain stable ball positioning.

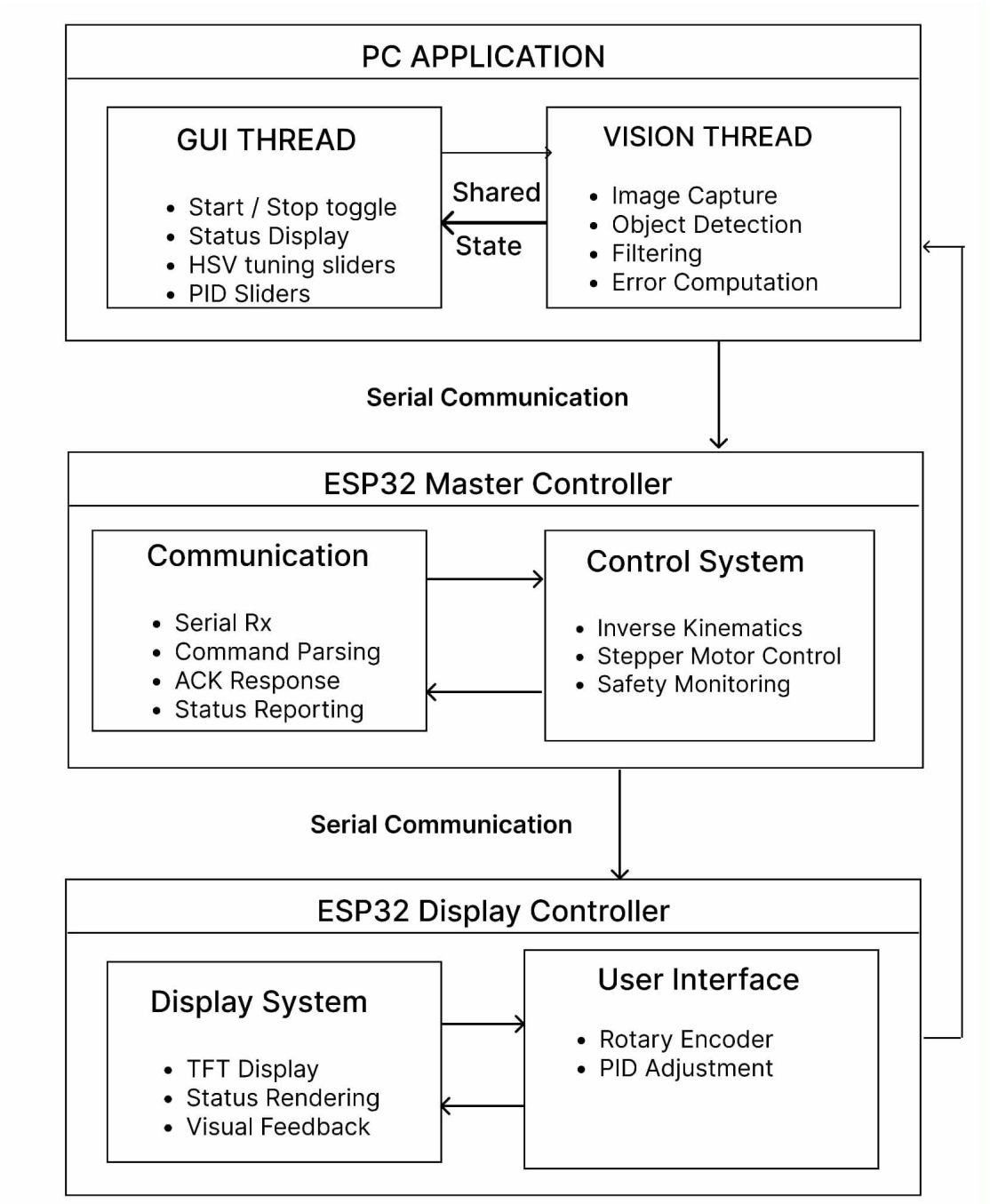

Figure 4.2: Overall System Architecture

4.4 Vision-Based Object Detection

A mobile phone camera connected to the PC via the DroidCam application continuously streams video frames of the platform. The video feed is processed using OpenCV, where image processing techniques such as color thresholding and contour detection are applied to identify the ball and extract its center coordinates.

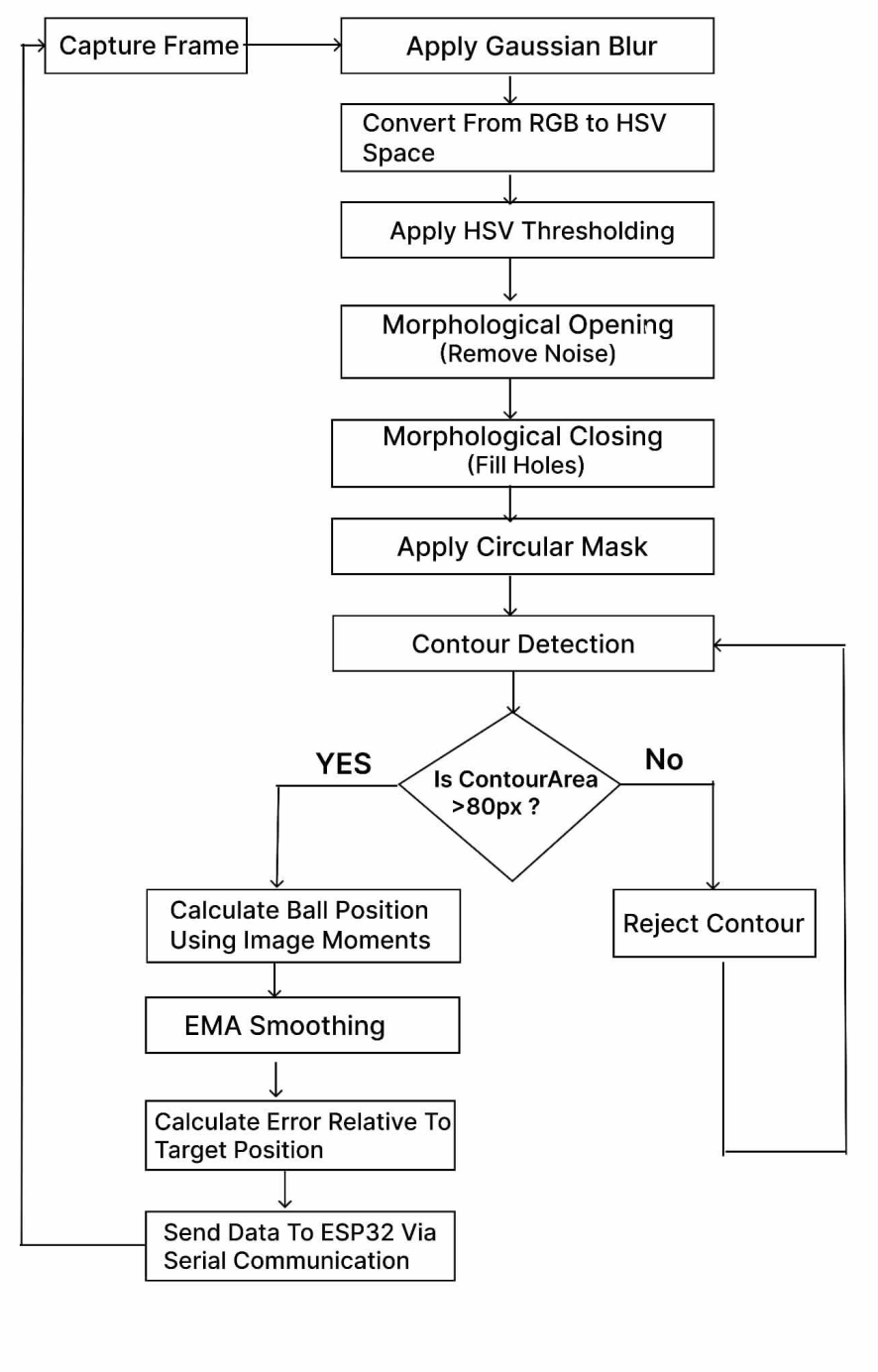

Figure 4.3: Flow diagram of the computer vision pipeline

4.5 PID Control Implementation

A classical PID controller is implemented to minimize the position error. The proportional term reacts to the current error, the integral term compensates for accumulated error, and the derivative term predicts future error trends. The PID output determines the magnitude and direction of platform tilt required to stabilize the ball. Controller

parameters are tuned experimentally based on system response. The PID controller is implemented in discrete time as:

$$u[k] = K_p e[k] + K_i \sum e[k] + K_d \frac{e[k] - e[k-1]}{\Delta t} \tag{4.1}$$

where:

- $e[k]$ is the error at time step k
- Δt is the sampling time

The controller output is converted into stepper motor commands to adjust the platform tilt. The final experimentally tuned PID parameters used in the system are:

$$K_p = 1.8, \quad K_i = 0.04, \quad K_d = 0.65$$

These values provided stable real-time balancing performance with reduced overshoot and minimal steady-state error.

4.6 Stepper Motor Control

The control output from the PID algorithm is translated into stepper motor commands. Desired platform tilt angles are converted into discrete step counts using known motor resolution and microstepping configuration. Stepper motor drivers execute these commands, providing precise and repeatable platform motion.

4.7 Implemented Real-Time Control Loop

The system operates in a continuous closed-loop manner with the following steps:

- Capture live video frames of the platform using a mobile phone camera streamed to the PC via DroidCam.
- Detect and track the ball position in real time using computer vision techniques.
- Compute the positional error by comparing the detected position with the desired target location.

- Process the error using an embedded PID controller to determine corrective action.
- Convert the controller output into stepper motor commands.
- Adjust the platform orientation accordingly and repeat the process continuously.

This loop enables dynamic stabilization of the ball even in the presence of external disturbances.

4.8 Experimental Validation

The system is tested under controlled disturbances to evaluate stability, response time, and accuracy. Performance is assessed by observing the system's ability to return the ball to the desired position and maintain balance over time.

4.9 Hardware Wiring and Integration

The complete hardware wiring of the system includes stepper motors, TB6600 motor drivers, ESP32 controllers, power supply units, TFT display, and rotary encoder. The vision input is obtained separately using a mobile phone camera connected wirelessly to the PC via DroidCam; therefore, no physical camera wiring is required in the embedded hardware setup.

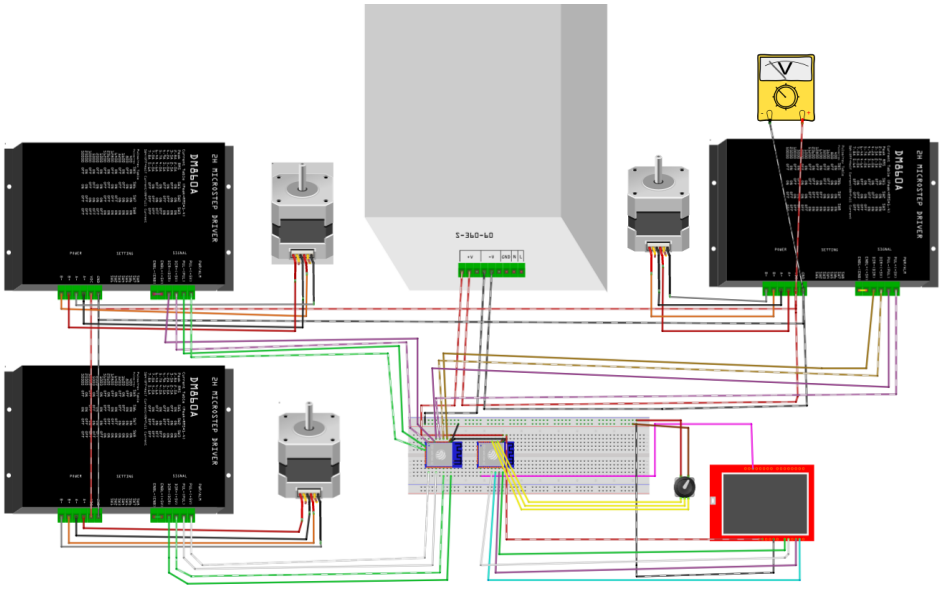

Figure 4.4: Detailed hardware wiring diagram

4.10 System Implementation

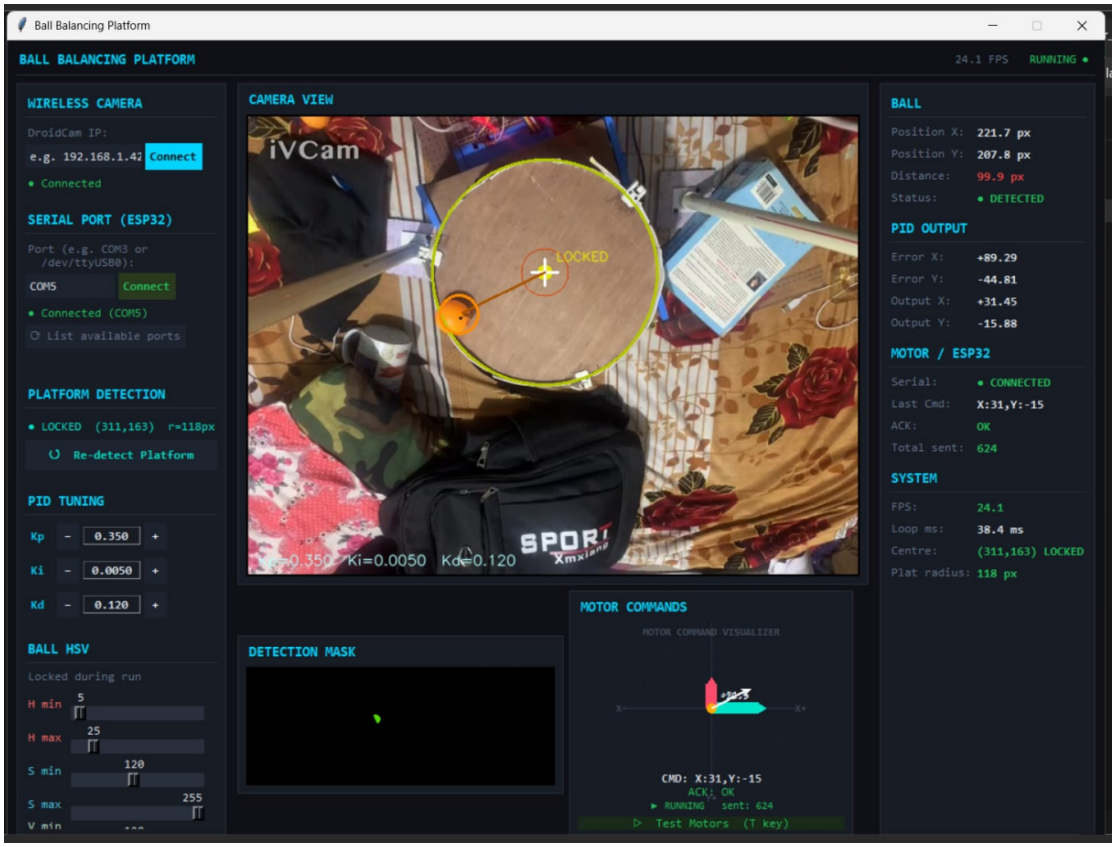

Figure 4.5: Real-time system interface

CHAPTER 5

RESULTS AND DISCUSSION

5.1 System Implementation Results

The proposed vision-based ball balancing system was successfully implemented and tested. The system integrates real-time computer vision with embedded PID control to stabilize a ball on a circular platform.

The system demonstrated the following functional capabilities:

- Real-time ball detection using OpenCV via mobile camera input
- Accurate position extraction and coordinate mapping
- Reliable UART communication between PC and ESP32
- Effective PID-based control for dynamic stabilization
- Smooth actuation using stepper motors with microstepping
- Real-time monitoring through TFT display interface

5.2 Performance Metrics

The system performance is evaluated using standard control system parameters.

5.2.1 Settling Time

Settling time is defined as the time required for the system output to remain within $\pm 2\%$ of the final value.

5.2.2 Overshoot

Overshoot is calculated as:

$$\%OS = \frac{(Peak\ Value - Final\ Value)}{Final\ Value} \times 100 \tag{5.1}$$

5.2.3 Steady-State Error

The steady-state error is given by:

$$e_{ss} = \lim_{t \to \infty} e(t) \tag{5.2}$$

5.3 Performance Evaluation

The system performance was evaluated using standard control system parameters:

5.3.1 Settling Time

- The system stabilizes the ball within 2–4 seconds after disturbance.
- Faster settling achieved after PID tuning.

5.3.2 Steady-State Error

- Minimal steady-state error observed.
- Ball remains close to target position with negligible deviation.

5.3.3 Overshoot

- Initial overshoot was observed during tuning.
- Reduced significantly after optimizing PID parameters.

5.3.4 Stability

- System remains stable under small disturbances.
- Maintains continuous balance in closed-loop operation.

5.3.5 Experimental Results

The system was tested under multiple trials to evaluate performance.

Table 5.1: System Performance Results

Trial	Settling Time (s)	Overshoot (%)	Steady-State Error (pixels)
1	3.1	9.0	3
2	2.8	8.5	2
3	3.0	8.7	2

5.3.6 Graphical Analysis

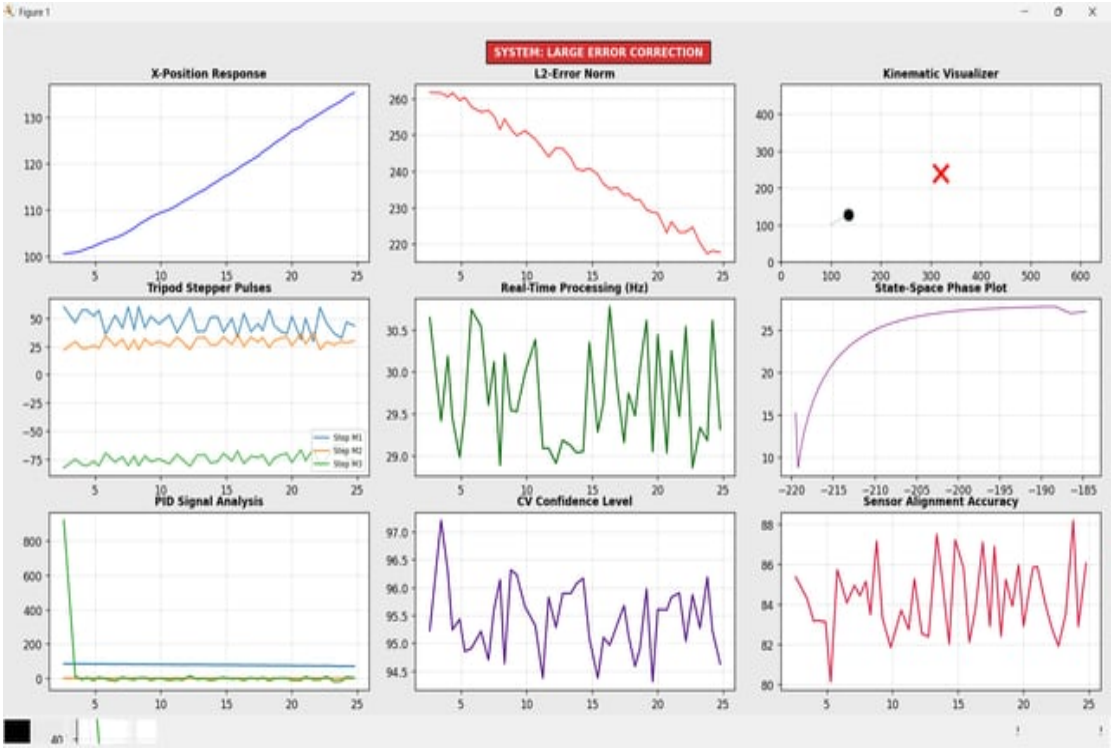

Figure 5.1: Large Error Correction

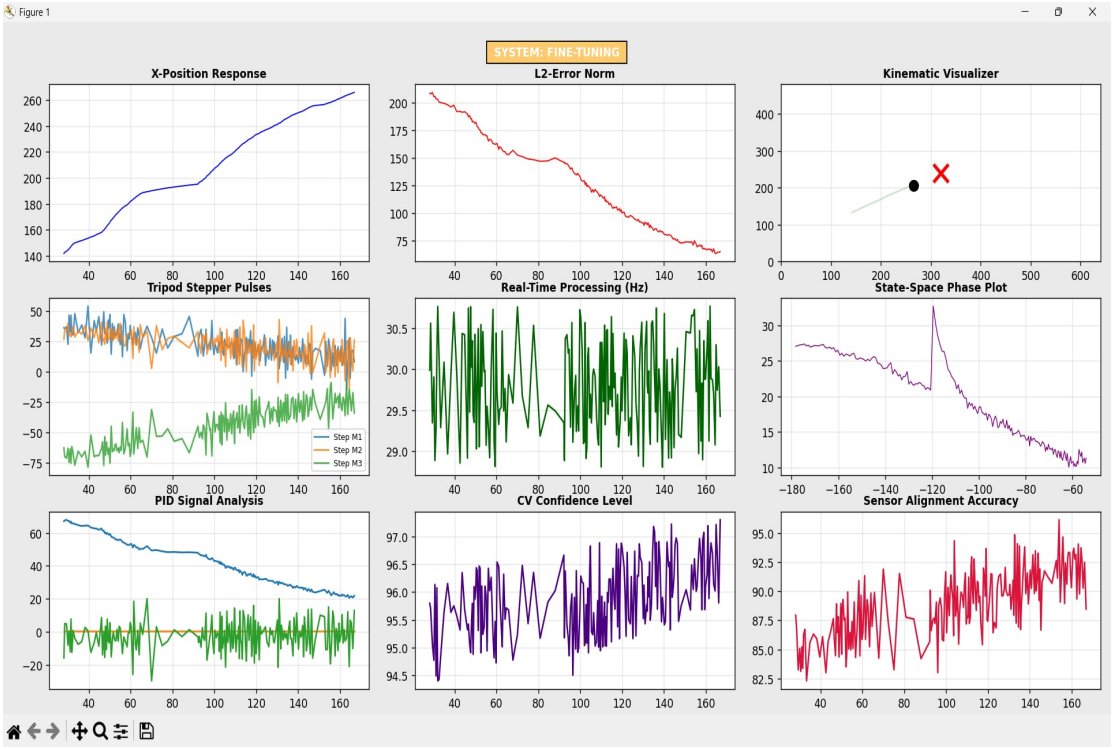

Figure 5.2: Fine Tuning

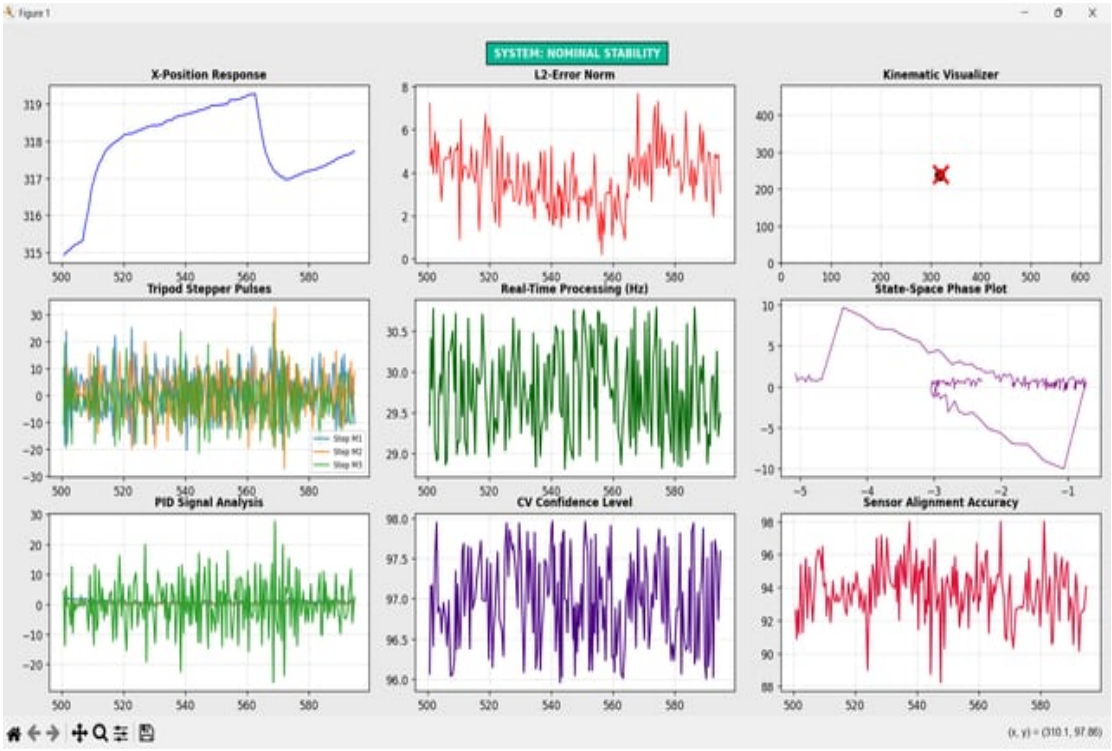

Figure 5.3: Normal Stability

The figure shows the real-time software analysis and performance monitoring of the vision-based ball balancing system. The plots represent important system parameters

such as X-position response, L2 error reduction, stepper motor pulse outputs, processing frequency, PID control signals, computer vision confidence level, sensor alignment accuracy, and system state-space behavior. The decreasing error curve indicates that the PID controller is successfully stabilizing the ball position over time. The processing frequency remains close to 30 Hz, confirming stable real-time operation. The visualizer and performance graphs demonstrate smooth control response, reliable object tracking, and accurate coordination between the computer vision module and embedded control system.

5.3.7 Vision System Performance

The vision system demonstrated reliable performance during real-time operation. The following observations were made:

- Ball detection accuracy was high under controlled lighting conditions.
- The frame processing rate was approximately 20–30 FPS, depending on hardware capabilities.
- Latency was sufficiently low to support real-time control.
- Noise in the visual data was effectively handled using:
  - Gaussian blur
  - Morphological operations

5.4 Control System Performance

The PID controller successfully minimized position error:

- Proportional term $\rightarrow$ Immediate response
- Integral term $\rightarrow$ Eliminates steady-state error
- Derivative term $\rightarrow$ Reduces oscillations

The control loop operates continuously and ensures stable feedback correction.

5.4.1 Hardware Performance

The hardware implementation of the system demonstrated reliable and efficient operation. The following key observations were made:

- Stepper motors provided precise and repeatable motion.
- TB6600 drivers enabled smooth microstepping, resulting in finer control of platform movement.
- The redesigned power system eliminated voltage drops and ensured stable operation.
- The dual ESP32 architecture improved overall system performance by:
  - Enhancing real-time performance
  - Enabling effective task separation
  - Increasing system reliability

5.5 System Limitations Observed

- Performance affected under poor lighting
- Camera latency introduces slight delay
- Manual PID tuning required
- Mechanical alignment affects accuracy

5.6 Challenges Faced

During the implementation of the system, several challenges were encountered:

- Sensitivity of computer vision to lighting conditions and camera alignment.
- Latency introduced by image processing and serial communication.
- Difficulty in tuning PID parameters for stable performance.
- Mechanical alignment issues affecting platform accuracy.
- Synchronization challenges between vision processing and embedded control.

CHAPTER 6

CONCLUSION

This project successfully demonstrated the design and implementation of a vision-based ball balancing platform using OpenCV and PID control. The system combines computer vision for real-time ball tracking, embedded control using ESP32, and precise stepper motor actuation for platform movement. A camera continuously captures the ball position, and the OpenCV module processes the image to detect the ball and calculate positional error. This error is sent to the ESP32 controller, where the PID algorithm generates corrective motor commands to stabilize the ball.

The developed platform operates as a closed-loop control system capable of maintaining stable ball positioning with real-time response. Experimental testing showed a settling time of approximately 2.8 seconds and an overshoot of 8.5%, indicating stable and responsive system behavior. Although a small steady-state error was observed due to camera alignment, mechanical limitations, and processing delay, the system achieved reliable balancing performance overall.

The project demonstrates that low-cost vision-based control systems can be practical, efficient, and suitable for educational, research, and real-time automation applications.

CHAPTER 7

FUTURE ENHANCEMENT

The current system can be further improved with the following enhancements:

7.1 Advanced Control Techniques

- Implement Adaptive PID or Model Predictive Control (MPC) for better stability and faster response.

7.2 AI-Based Vision System

- Replace color-based tracking with CNN-based object detection for improved accuracy under varying lighting conditions.

7.3 Hardware and System Optimization

- Use faster actuators and improved synchronization techniques to enhance real-time system performance and support industrial automation applications.

REFERENCES

[1] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.

[2] K. Ogata, *Modern Control Engineering*, 5th ed. Prentice Hall, 2010.

[3] L. Zhao, H. Wang, and Y. Liu, "Vision-based control for ball-balancing robots: A review," *Journal of Intelligent & Robotic Systems*, vol. 101, no. 3, pp. 45–59, 2021.

[4] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.

[5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[6] J. Shi and C. Tomasi, "Good features to track," in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1994, pp. 593–600.

[7] J. Y. Bouguet, "Pyramidal implementation of the Lucas-Kanade feature tracker," Intel Corporation, Tech. Rep., 2001.

[8] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed. Springer, 2017.

[9] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*, 2nd ed. Springer, 2016.

[10] J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2017.

[11] Y. Wang and M. Chen, "Real-time object tracking and PID-based balancing system using computer vision," *International Journal of Advanced Robotic Systems*, vol. 18, no. 2, pp. 1–12, 2021.

[12] D. E. Kirk, *Optimal Control Theory: An Introduction*. Dover Publications, 2004.

[13] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson Education, 2009.

[14] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[15] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

APPENDIX A:Project Budget

Item	Quantity	Unit Cost (NPR)	Total Cost (NPR)
ESP32 Development Board	2	1300	2600
NEMA 17 Stepper Motor	3	1800	5400
TB6600 Stepper Motor Driver	3	1500	4500
TFT Display	1	2000	2000
Rotary Encoder	1	600	600
12V SMPS Power Supply	1	1500	1500
Buck Converter Modules	2	500	1000
3D Printed Platform	1	1500	1500
Miscellaneous (Wires, connectors, mounts)	-	1500	1500
Total	-	-	**20,600**

Table A.1: Revised Project Budget

APPENDIX B: Core Python Implementation

```python
CONFIG = {
    # PID defaults — conservative safe starting values for stepper motors
    "pid_kp":           0.05,
    "pid_ki":           0.00,
    "pid_kd":           0.0,
    "pid_output_limit":   50.0,   # max tilt command sent to ESP32
    "pid_integral_limit": 40.0,   # anti-windup: clamp the integral term

    # PID tuning step sizes (per button press)
    "pid_kp_step":      0.05,
    "pid_ki_step":      0.05,
    "pid_kd_step":      0.05,

    # PID upper caps (prevent runaway tuning)
    "pid_kp_max":       20.0,
    "pid_ki_max":       20.0,
    "pid_kd_max":       20.0,

    # Ball position smoothing (EMA filter)
    "ema_alpha":        0.35,   # 0 = very smooth (laggy), 1 = raw (jittery)

    # Camera
    "camera_index":     1,      # DroidCam or webcam index
    "frame_width":      640,
    "frame_height":     480,
    "target_fps":       30,
    "warmup_frames":    30,     # skip first N frames (camera auto-exposure settle)

    # HSV — orange ball detection
    "hsv_lower":        [5,  100, 100],
    "hsv_upper":        [25, 255, 255],

    # Detection sizes
    "min_ball_area":    200,    # ignore blobs smaller than this (pixels²)
    "platform_radius_frac": 0.42,  # platform circle = 42% of frame size
    "morph_open_k":     5,      # kernel size for noise removal
    "morph_close_k":    11,     # kernel size for gap filling



    # Serial (ESP32)
    "serial_port":      "COM3",
    "serial_baud":      115200,
    "serial_timeout":   0.05,
    "serial_send_every": 3,     # send command every Nth frame (avoid flooding)
    "serial_reconnect_interval": 2.0,

    # Safety
    "max_lost_frames": 20,      # reset PID after losing ball this many frames

    # Visualisation
    "trajectory_len":   60,     # how many past positions to draw as trail
    "tolerance_px":     25,     # green circle radius (ball "in zone" threshold)
}
```

Figure B.1: System Configuration

```
# Step 2: convert to HSV colour space
hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)
# Step 3: create colour mask (white = ball, black = background)
mask = cv2.inRange(hsv, lower, upper)
```

Figure B.2: HSV Ball Detection

```
# Step 4: remove small noise blobs
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  self._open_k)
# Step 5: fill small gaps in the blob
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, self._close_k)
```

Figure B.3: Noise Removal

```
# Step 6: restrict to ROI rectangle
mask = cv2.bitwise_and(mask, self._make_roi_mask(state))
```

Figure B.4: ROI-Based Detection

```
M = cv2.moments(largest)
if M["m00"] == 0:
    return None, None, None, mask

raw_x = M["m10"] / M["m00"]
raw_y = M["m01"] / M["m00"]
```

Figure B.5: Ball Position Using Moments

```
# Step 9: smooth position using EMA filter
if self._smooth_x is None:
    self._smooth_x, self._smooth_y = raw_x, raw_y
else:
    self._smooth_x = alpha * raw_x + (1 - alpha) * self._smooth_x
    self._smooth_y = alpha * raw_y + (1 - alpha) * self._smooth_y

return self._smooth_x, self._smooth_y, float(radius), mask
```

Figure B.6: EMA Smoothing

```
error_x = ball_x - sx
error_y = ball_y - sy
```

Figure B.7: Error Calculation

```python
# P term: proportional to current error
p = kp * error

# I term: accumulated error over time (with anti-windup clamp)
self._integral += error * dt
lim = CONFIG["pid_integral_limit"]
self._integral = float(np.clip(self._integral, -lim, lim))
i = ki * self._integral

# D term: rate of change of error
d = kd * (error - self._prev_error) / dt
self._prev_error = error

output = p + i + d
```

Figure B.8: PID Controller

```python
msg = f"X:{int(x)},Y:{int(y)}\n"
with self._lock:
    try:
        self._ser.write(msg.encode())
```

Figure B.9: Serial Communication

```python
if lost >= CONFIG["max_lost_frames"]:
    # Ball lost for too long — reset PID to avoid integral
    self._pid_x.reset()
    self._pid_y.reset()
    self._detector.reset_smooth()
    self._serial.send_safe()
```

Figure B.10: Ball Lost Safety

APPENDIX C:ESP32 Control Implementation

```
while (Serial.available() > 0) {
  char c = (char)Serial.read();
  input += c;
```

Figure C.1: Serial Data Receiving

```
// Find "X:" prefix
char* x_ptr = strstr(rx_buf, "X:");
char* y_ptr = strstr(rx_buf, "Y:");

if (x_ptr != NULL && y_ptr != NULL) {
  parsed_x = atoi(x_ptr + 2);   // skip "X:"
  parsed_y = atoi(y_ptr + 2);   // skip "Y:"
```

Figure C.2: Command Parsing

```
parsed_x = constrain(parsed_x, (int)(-MAX_TILT_INPUT), (int)(MAX_TILT_INPUT));
parsed_y = constrain(parsed_y, (int)(-MAX_TILT_INPUT), (int)(MAX_TILT_INPUT));
```

Figure C.3: Input Limiting

```
float delta_h = mx * sin(pitch) + my * sin(roll);

// Arm angle needed to achieve this height
// Clamp the ratio to [-1, 1] before arcsin to avoid NaN
float ratio = delta_h / ARM_LENGTH_MM;
ratio = constrain(ratio, -0.99f, 0.99f);
float arm_angle_rad = asin(ratio);

// Convert arm angle to motor steps
// Full revolution = TOTAL_STEPS_PER_REV steps = 2π radians
steps[i] = (long)(arm_angle_rad * TOTAL_STEPS_PER_REV / (2.0f * PI));
```

Figure C.4: Inverse Kinematics

```
void commandMotors(long s1, long s2, long s3) {
  motor1.moveTo(s1);
  motor2.moveTo(s2);
  motor3.moveTo(s3);
}
```

Figure C.5: Motor Commanding

```
if ((now - last_valid_msg_ms) > WATCHDOG_MS) {
  if (!watchdog_active) {
    // Watchdog just fired — initiate safe home
    watchdog_active = true;
    cmd_x = 0.0f;
    cmd_y = 0.0f;
    goHomeGently();
```

Figure C.6: Watchdog Safety

```
void loop() {

  // ── 1. Read serial (non-blocking, takes latest) ──────────────
  parseLatestSerial();

  // ── 2. Watchdog check ────────────────────────────────────────
  checkWatchdog();

  // ── 3. Apply latest command (only if changed) ────────────────
  // Skip IK calculation if command has not changed since last loop
  // This saves CPU cycles (IK uses sin/cos/asin which are expensive)
  if (!watchdog_active) {
    if (cmd_x != last_applied_x || cmd_y != last_applied_y) {
      commandMotorsFromTilt(cmd_x, cmd_y);
      last_applied_x = cmd_x;
      last_applied_y = cmd_y;
    }
  }
  // When watchdog is active: goHomeGently() already set targets to 0
  // Just let stepper.run() below bring motors home gradually

  // ── 4. Run steppers — MUST be called every loop iteration ────
  // AccelStepper needs run() called as frequently as possible.
  // Each call moves the motor by at most one step if it is due.
  // Not calling this frequently enough = missed steps = jitter = wrong position
  motor1.run();
  motor2.run();
  motor3.run();

  // ── NO delay here ────────────────────────────────────────────
  // Even delayMicroseconds(1) would reduce max step rate.
  // The loop runs as fast as ESP32 allows (~100,000+ times/sec).
  // This is intentional and correct.
}
```

Figure C.7: Main Loop